

École Nationale Supérieure d'Informatique et
d'Analyse des Systèmes
ENSIAS

Rapport Technique d'Avancement

DevPilot AI

Plateforme Agentic RAG pour l'Intelligence
Documentaire Privée

Documentation des Phases 12 à 15

Évaluation, Workflow Agentique, Recherche Hybride et Reranking

Réalisé par : Oussama Mahi MALIH Bilal

Projet : DevPilot AI

Domaine : Backend Engineering, RAG, Agentic AI, Information Retrieval

Stack principale : FastAPI, PostgreSQL, Redis, Celery, Qdrant, Ollama, Docker

Table des matières

Résumé exécutif	4
1 Contexte général du projet	5
1.1 Objectif de DevPilot AI	5
1.2 État du système avant la Phase 12	5
1.3 Objectif des phases 12 à 15	6
2 Architecture globale des phases 12 à 15	7
2.1 Vue d'ensemble	7
2.2 Pourquoi cette architecture ?	9
3 Phase 12 : Évaluation automatique des réponses	10
3.1 Objectif de la Phase 12	10
3.2 Pourquoi évaluer les réponses ?	10
3.3 Métriques utilisées	10
3.3.1 Faithfulness	10
3.3.2 Relevance	11
3.3.3 Context precision	11
3.3.4 Hallucination score	11
3.4 Implémentation	11
3.5 Exemple de réponse évaluée	12
3.6 Fallback heuristique	12
3.7 Test manuel de la Phase 12	12
3.7.1 Appel du chat avec évaluation	12
3.7.2 Récupération de l'évaluation par message	13
3.7.3 Vérification dans PostgreSQL	13
3.8 Validation de la Phase 12	13
4 Phase 13 : Workflow agentique	14
4.1 Objectif de la Phase 13	14
4.2 Pourquoi utiliser un workflow agentique ?	14
4.3 Agents implémentés	14
4.3.1 RouterAgent	14
4.3.2 QueryRewriterAgent	15
4.3.3 RetrievalAgent	15
4.3.4 GeneratorAgent	15
4.3.5 EvaluatorAgent	15
4.3.6 CorrectorAgent	15
4.4 Traçabilité avec AgentRun	16

4.5	Exemple de workflow validé	16
4.6	Test manuel de la Phase 13	16
4.6.1	Question technique	16
4.6.2	Vérification des agents	17
4.7	Validation de la Phase 13	17
5	Phase 14 : Recherche hybride	18
5.1	Objectif de la Phase 14	18
5.2	Pourquoi utiliser une recherche hybride ?	18
5.3	Recherche sémantique	18
5.4	Recherche par mots-clés	18
5.5	Fusion par RRF	19
5.6	Endpoint de recherche hybride	19
5.7	Utilisation dans le chat	19
5.8	Résultat obtenu	20
5.9	Validation de la Phase 14	20
6	Phase 15 : Reranking	21
6.1	Objectif de la Phase 15	21
6.2	Pourquoi utiliser le reranking ?	21
6.3	Reranking heuristique	21
6.4	Workflow du reranking	22
6.5	Agent Reranker	22
6.6	Exemple validé	22
6.7	Sortie du reranker	22
6.8	Test manuel de la Phase 15	23
6.8.1	Question avec Celery	23
6.8.2	Vérification des chunks utilisés	23
6.8.3	Vérification de l'agent reranker	23
6.8.4	Vérification de la sortie du reranker	24
6.9	Validation de la Phase 15	24
7	Résultats techniques observés	25
7.1	État des phases	25
7.2	Latences observées	25
7.3	Analyse des performances	25
8	Pourquoi ces technologies sont utilisées	26
8.1	FastAPI	26
8.2	PostgreSQL	26
8.3	Qdrant	26
8.4	Ollama	27
8.5	Redis et Celery	27
8.6	Recherche hybride	27
8.7	Reranking	27

9	Commandes de démonstration	28
9.1	Tester l'évaluation	28
9.2	Tester les agents	28
9.3	Tester la recherche hybride	28
9.4	Tester le reranking	28
10	Difficultés rencontrées et améliorations	30
10.1	Problème du corrector trop agressif	30
10.2	Fallback heuristique	30
10.3	Latence du modèle	30
11	Conclusion	31

Résumé exécutif

Ce rapport présente l'évolution du projet **DevPilot AI** durant les phases 12 à 15. Ces phases représentent une étape importante dans la professionnalisation du système, car elles ajoutent des mécanismes avancés permettant d'améliorer la fiabilité, la traçabilité et la qualité des réponses générées.

Après les phases 9 à 11, le système possédait déjà un pipeline RAG fonctionnel : génération d'embeddings avec Ollama, stockage vectoriel dans Qdrant, recherche sémantique, génération de réponses avec citations et sauvegarde des conversations. Les phases 12 à 15 ajoutent une couche d'intelligence supplémentaire autour de ce pipeline.

La **Phase 12** introduit l'évaluation automatique des réponses afin de mesurer la fiabilité, la pertinence, la précision du contexte et le risque d'hallucination. La **Phase 13** transforme le pipeline RAG classique en un workflow agentique composé de plusieurs agents spécialisés. La **Phase 14** ajoute la recherche hybride combinant la recherche sémantique et la recherche par mots-clés. La **Phase 15** introduit le reranking, qui réordonne les chunks récupérés pour sélectionner les meilleurs contextes avant la génération.

À la fin de ces phases, DevPilot AI devient une plateforme RAG avancée, capable de rechercher, générer, évaluer, corriger et tracer ses décisions internes.

1 Contexte général du projet

1.1 Objectif de DevPilot AI

DevPilot AI est une plateforme sécurisée de type *Agentic RAG* destinée à l'exploitation intelligente de documents privés. Elle permet à un utilisateur de téléverser des documents, de les indexer, puis de poser des questions en langage naturel afin d'obtenir des réponses basées uniquement sur ses documents.

Le projet vise à démontrer des compétences avancées en :

- ingénierie backend ;
- conception de bases de données ;
- traitement asynchrone ;
- recherche vectorielle ;
- RAG ;
- sécurité applicative ;
- évaluation de réponses IA ;
- orchestration agentique ;
- optimisation de la recherche documentaire.

1.2 État du système avant la Phase 12

Avant la Phase 12, DevPilot AI permettait déjà :

- l'authentification des utilisateurs par JWT ;
- la gestion de workspaces privés ;
- l'upload de documents PDF, TXT et Markdown ;
- le traitement asynchrone avec Celery et Redis ;
- le nettoyage et le chunking des textes ;
- la génération d'embeddings avec Ollama ;
- l'indexation vectorielle dans Qdrant ;
- la recherche sémantique ;
- la génération de réponses avec citations.

Cependant, le système ne disposait pas encore d'un mécanisme permettant d'évaluer automatiquement la qualité des réponses, de tracer les étapes internes du raisonnement applicatif, ni d'améliorer la qualité du contexte récupéré par combinaison de plusieurs stratégies de recherche.

1.3 Objectif des phases 12 à 15

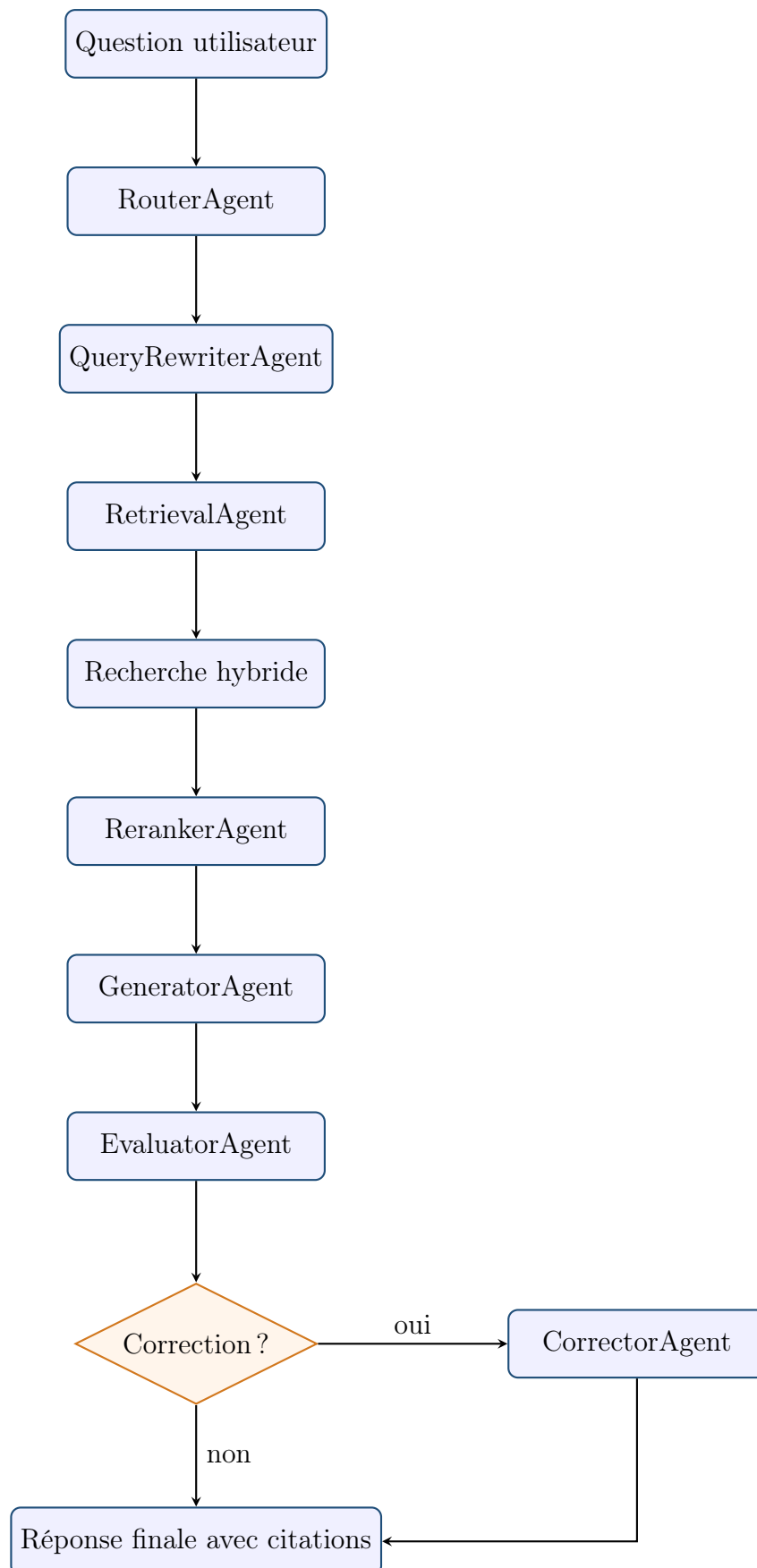
Les phases 12 à 15 ajoutent des composants avancés autour du pipeline RAG :

Phase	Objectif principal
Phase 12	Évaluer automatiquement la qualité des réponses générées.
Phase 13	Introduire un workflow agentique composé d'agents spécialisés.
Phase 14	Combiner la recherche sémantique et la recherche par mots-clés.
Phase 15	Réordonner les chunks récupérés afin d'améliorer le contexte envoyé au modèle.

2 Architecture globale des phases 12 à 15

2.1 Vue d'ensemble

L'architecture avancée du pipeline peut être représentée comme suit :



2.2 Pourquoi cette architecture ?

Un pipeline RAG simple peut fonctionner pour des cas basiques, mais il peut échouer dans plusieurs situations :

- récupération de chunks peu pertinents ;
- réponse correcte mais non évaluée ;
- risque d'hallucination ;
- absence de traçabilité ;
- mauvaise gestion des questions hors contexte ;
- faible qualité de réponse lorsque le contexte est trop large ou mal classé.

L'approche des phases 12 à 15 améliore ces aspects en ajoutant :

- une évaluation automatique ;
- des agents spécialisés ;
- une recherche hybride ;
- un reranking des chunks ;
- une traçabilité complète dans PostgreSQL.

3 Phase 12 : Évaluation automatique des réponses

3.1 Objectif de la Phase 12

La Phase 12 introduit un système d'évaluation automatique des réponses générées par le pipeline RAG.

L'objectif n'est pas seulement de générer une réponse, mais aussi de mesurer sa qualité selon plusieurs critères :

- la fidélité au contexte ;
- la pertinence par rapport à la question ;
- la précision du contexte récupéré ;
- le risque d'hallucination.

Cette phase est essentielle car un système RAG professionnel doit être capable de s'auto-évaluer ou, au minimum, de fournir des métriques permettant d'analyser ses performances.

3.2 Pourquoi évaluer les réponses ?

Dans une application RAG, le modèle de langage peut produire une réponse qui semble correcte mais qui n'est pas réellement supportée par les documents. C'est ce qu'on appelle une hallucination.

L'évaluation permet de répondre à des questions importantes :

- La réponse est-elle basée sur les documents ?
- La réponse répond-elle réellement à la question ?
- Les chunks récupérés étaient-ils utiles ?
- Le modèle a-t-il inventé une information ?

3.3 Métriques utilisées

3.3.1 Faithfulness

La métrique **faithfulness** mesure le degré de fidélité de la réponse par rapport au contexte récupéré.

Une valeur élevée signifie que la réponse est bien supportée par les chunks.

```
faithfulness = 0.85
```

Interprétation :

- proche de 1 : réponse bien fondée ;
- proche de 0 : réponse probablement inventée ou non supportée.

3.3.2 Relevance

La métrique **relevance** mesure la pertinence de la réponse par rapport à la question posée.

Par exemple, pour la question :

```
What technologies does DevPilot AI use?
```

Une bonne réponse doit mentionner les technologies utilisées :

```
FastAPI, PostgreSQL, Redis, Celery, Ollama, Qdrant.
```

3.3.3 Context precision

La métrique **context_precision** indique si les chunks récupérés sont utiles pour répondre à la question.

Une valeur de 1.0 indique que les chunks utilisés sont pertinents pour la question.

3.3.4 Hallucination score

La métrique **hallucination_score** mesure le risque d'hallucination.

Une faible valeur est souhaitable.

```
hallucination_score = 0.05
```

signifie que le risque d'hallucination est très faible.

3.4 Implémentation

L'évaluation est déclenchée après la génération de la réponse. Le pipeline devient donc :

```
Question utilisateur
-> Retrieval
-> Generation
-> Evaluation
-> Sauvegarde des scores
-> Retour de la reponse avec evaluation
```

L'évaluation est sauvegardée dans la table :

```
evaluations
```

Chaque ligne est liée à un message assistant via :

```
message_id
```

3.5 Exemple de réponse évaluée

Une réponse obtenue pendant les tests était :

```
DevPilot AI uses FastAPI, PostgreSQL, Redis, Celery, Ollama, and Qdrant [1],  
[2], [3].
```

L'évaluation retournée était :

```
faithfulness: 0.85  
relevance: 0.90  
context_precision: 1.0  
hallucination_score: 0.25
```

Cette évaluation indique que la réponse est pertinente, correctement fondée sur le contexte et présente un risque d'hallucination modéré à faible.

3.6 Fallback heuristique

Lorsque l'évaluation par LLM n'est pas disponible, le système utilise un fallback heuristique.

Ce fallback permet de continuer à produire une évaluation même si le modèle d'évaluation n'est pas disponible ou prend trop de temps.

Exemple d'explication :

```
Heuristic fallback used because LLM evaluation was unavailable.
```

Ce mécanisme est important pour la robustesse du système. Il évite qu'une indisponibilité du modèle d'évaluation bloque toute la réponse.

3.7 Test manuel de la Phase 12

3.7.1 Appel du chat avec évaluation

```
RESPONSE=$(curl -fsS -X POST http://localhost:8000/api/v1/workspaces/  
$WORKSPACE_ID/chat/query \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{"question":"What technologies does DevPilot AI use?}")  
  
echo "$RESPONSE" | jq '{answer,citations,evaluation,message_id}'
```

Résultat attendu :

```
answer  
citations  
evaluation  
message_id
```

3.7.2 Récupération de l'évaluation par message

```
MESSAGE_ID=$(echo "$RESPONSE" | jq -r .message_id)

curl -fsS http://localhost:8000/api/v1/messages/$MESSAGE_ID/evaluation \
-H "Authorization: Bearer $TOKEN" | jq
```

Résultat attendu :

```
faithfulness
relevance
context_precision
hallucination_score
explanation
```

3.7.3 Vérification dans PostgreSQL

```
docker compose exec postgres psql -U devpilot -d devpilot -c \
"select message_id, faithfulness, relevance, context_precision,
   hallucination_score
from evaluations
order by created_at desc
limit 5;"
```

3.8 Validation de la Phase 12

La Phase 12 est validée si :

- une évaluation est retournée avec la réponse ;
- l'évaluation est sauvegardée dans PostgreSQL ;
- l'évaluation peut être récupérée par `message_id` ;
- les métriques sont cohérentes ;
- les questions hors contexte sont évaluées avec un faible risque d'hallucination si le modèle refuse correctement.

4 Phase 13 : Workflow agentique

4.1 Objectif de la Phase 13

La Phase 13 transforme le pipeline RAG classique en un pipeline agentique. Au lieu d'avoir une seule chaîne linéaire, le système est divisé en plusieurs agents spécialisés.

Chaque agent a une responsabilité précise :

- router la question ;
- reformuler la requête ;
- récupérer les chunks ;
- générer la réponse ;
- évaluer la réponse ;
- corriger la réponse si nécessaire.

4.2 Pourquoi utiliser un workflow agentique ?

Un pipeline RAG simple peut être difficile à contrôler. Le workflow agentique apporte :

- une meilleure séparation des responsabilités ;
- une meilleure traçabilité ;
- une facilité de debugging ;
- une meilleure extensibilité ;
- la possibilité d'ajouter de nouveaux agents plus tard.

Cette approche rapproche DevPilot AI d'une architecture de type *AI Platform* professionnelle.

4.3 Agents implémentés

4.3.1 RouterAgent

Le **RouterAgent** analyse la question utilisateur et décide du type de requête.

Exemples de types :

- `factual_question` ;
- `summary_question` ;
- `comparison_question` ;

— `technical_question`.

Il peut également choisir une stratégie de retrieval :

```
semantic  
keyword  
hybrid
```

4.3.2 QueryRewriterAgent

Le `QueryRewriterAgent` reformule la question si nécessaire.

Par exemple, une question vague comme :

```
Explain it
```

peut être reformulée ou traitée avec prudence pour éviter une mauvaise interprétation.

4.3.3 RetrievalAgent

Le `RetrievalAgent` appelle le service de recherche pour récupérer les chunks pertinents.

Il peut utiliser différentes stratégies :

- recherche sémantique ;
- recherche par mots-clés ;
- recherche hybride.

4.3.4 GeneratorAgent

Le `GeneratorAgent` construit le prompt RAG et appelle Ollama pour générer la réponse finale.

Le modèle utilisé peut être :

```
qwen3:8b
```

pour une meilleure qualité, ou un modèle plus petit pour un développement plus rapide.

4.3.5 EvaluatorAgent

Le `EvaluatorAgent` calcule les métriques de qualité :

- `faithfulness` ;
- `relevance` ;
- `context precision` ;
- `hallucination score`.

4.3.6 CorrectorAgent

Le `CorrectorAgent` intervient lorsque la réponse semble problématique.

Il peut :

- conserver la réponse si elle est correcte ;
- remplacer la réponse si le contexte est insuffisant ;
- éviter de diffuser une réponse hallucinée.

4.4 Traçabilité avec AgentRun

Chaque agent est sauvegardé dans la table :

```
agent_runs
```

Chaque entrée contient :

- le type d'agent ;
- le statut ;
- l'entrée ;
- la sortie ;
- la latence ;
- le message associé.

4.5 Exemple de workflow validé

Pour une question hors contexte :

```
What is the CEO favorite food?
```

Le système a exécuté :

```
router
query_rewriter
retrieval
generator
evaluator
corrector
```

Le résultat final était :

```
I could not find this information in the uploaded documents.
```

Ce comportement est correct, car l'information n'existait pas dans les documents.

4.6 Test manuel de la Phase 13

4.6.1 Question technique

```
RESPONSE=$(curl -fsS -X POST http://localhost:8000/api/v1/workspaces/
  $WORKSPACE_ID/chat/query \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"question":"What technologies does DevPilot AI use?"}')

echo "$RESPONSE" | jq '{answer,citations,evaluation,message_id}'
```

4.6.2 Vérification des agents

```
MESSAGE_ID=$(echo "$RESPONSE" | jq -r .message_id)

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select agent_type, status, latency_ms
from agent_runs
where message_id = '$MESSAGE_ID'
order by created_at;"
```

Résultat attendu :

```
router
query_rewriter
retrieval
generator
evaluator
corrector
```

4.7 Validation de la Phase 13

La Phase 13 est validée si :

- le chat continue à fonctionner ;
- tous les agents sont exécutés ;
- les exécutions sont sauvegardées dans `agent_runs` ;
- les latences sont enregistrées ;
- les questions hors contexte sont refusées proprement ;
- la réponse finale reste accompagnée de citations et d'une évaluation.

5 Phase 14 : Recherche hybride

5.1 Objectif de la Phase 14

La Phase 14 ajoute la recherche hybride. L'objectif est de combiner deux approches complémentaires :

- la recherche sémantique avec Qdrant ;
- la recherche par mots-clés avec PostgreSQL.

5.2 Pourquoi utiliser une recherche hybride ?

La recherche sémantique est efficace pour comprendre le sens d'une question. Cependant, elle peut parfois manquer des termes exacts importants.

La recherche par mots-clés est efficace pour les noms précis :

- Qdrant ;
- Celery ;
- Redis ;
- FastAPI ;
- PostgreSQL ;
- Ollama.

La recherche hybride combine les deux afin d'obtenir de meilleurs résultats.

5.3 Recherche sémantique

La recherche sémantique transforme la question en embedding, puis cherche les vecteurs les plus proches dans Qdrant.

Exemple :

document processing system

peut retrouver des chunks parlant de Celery, Redis et traitement documentaire même si les mots exacts sont différents.

5.4 Recherche par mots-clés

La recherche par mots-clés cherche des termes exacts dans les chunks stockés dans PostgreSQL.

Exemple :

```
Qdrant Celery
```

permet de retrouver les chunks contenant explicitement ces technologies.

5.5 Fusion par RRF

La fusion est réalisée avec la méthode *Reciprocal Rank Fusion*, ou RRF.

La formule est :

$$score = \sum \frac{1}{k + rank}$$

où :

- **rank** est le rang du document dans une stratégie donnée ;
- **k** est une constante, souvent fixée à 60.

Les scores RRF sont généralement petits, par exemple :

```
0.032786  
0.03125  
0.015873
```

Cela est normal, car ces scores représentent une fusion de rangs et non une similarité cosinus.

5.6 Endpoint de recherche hybride

```
curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/retrieval/  
search \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "query": "How does DevPilot use Qdrant and Celery?",  
  "top_k": 5,  
  "strategy": "hybrid"  
}
```

Résultat attendu :

```
retrieval_strategy = hybrid  
source_scores.semantic  
source_scores.keyword
```

5.7 Utilisation dans le chat

La stratégie hybride peut aussi être utilisée directement dans le chat :

```
curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/chat/query \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "question": "How does DevPilot use Qdrant and Celery?",
  "retrieval_strategy": "hybrid"
}'
```

5.8 Résultat obtenu

Le système a correctement répondu :

```
DevPilot AI uses Celery for asynchronous document processing.
Qdrant is used for vector search, where it stores embeddings generated by Ollama
.
```

La table `retrieved_chunks` a confirmé que la stratégie sauvegardée était :

```
hybrid
```

5.9 Validation de la Phase 14

La Phase 14 est validée si :

- la recherche sémantique fonctionne ;
- la recherche par mots-clés fonctionne ;
- la recherche hybride fonctionne ;
- les résultats hybrides contiennent `source_scores` ;
- le chat peut utiliser la stratégie hybride ;
- la stratégie `hybrid` est sauvegardée dans `retrieved_chunks`.

6 Phase 15 : Reranking

6.1 Objectif de la Phase 15

La Phase 15 ajoute une étape de reranking. Après la récupération initiale de plusieurs chunks, le système les réordonne pour sélectionner ceux qui sont les plus utiles à la génération.

L'objectif est d'envoyer au modèle les meilleurs contextes possibles.

6.2 Pourquoi utiliser le reranking ?

La recherche initiale peut retourner plusieurs chunks pertinents, mais leur ordre n'est pas toujours optimal.

Le reranking permet de :

- favoriser les chunks contenant les termes exacts de la question ;
- améliorer la qualité du contexte envoyé au modèle ;
- réduire le bruit ;
- améliorer la précision des réponses ;
- aider le modèle à citer les meilleures sources.

6.3 Reranking heuristique

Le reranking implémenté est basé sur plusieurs signaux :

- le score original de retrieval ;
- le recouvrement lexical entre la question et le chunk ;
- le nombre de correspondances exactes ;
- le rang original ;
- la présence de termes techniques importants.

Exemples de termes techniques :

Celery, Qdrant, Redis, FastAPI, PostgreSQL, Ollama
--

6.4 Workflow du reranking

Le pipeline devient :

```
Question utilisateur
-> Retrieval candidates
-> Reranking
-> Selection top chunks
-> Generation
-> Evaluation
-> Correction
```

6.5 Agent Reranker

Le reranking est intégré dans le workflow agentique sous forme d'un agent :

```
reranker
```

Il est sauvegardé dans `agent_runs`, avec :

- les chunks candidats ;
- les scores originaux ;
- les nouveaux scores de reranking ;
- les features utilisées.

6.6 Exemple validé

Pour la question :

```
How does Celery process documents asynchronously?
```

Le système a retourné une réponse correcte :

```
Celery processes documents asynchronously by handling background jobs through
the backend APIs, allowing document processing tasks such as text extraction
, cleaning, chunking, and embedding to occur in the background without
blocking the main application.
```

L'agent `reranker` a été exécuté et enregistré.

6.7 Sortie du reranker

Un exemple de sortie du reranker contient :

```
rerank_score
overlap
exact_matches
original_rank
original_score
```

Exemple :

```
rerank_score = 0.988938
overlap = 0.8
exact_matches = 4
original_rank = 2
```

Cela signifie que le chunk était fortement pertinent pour la question.

6.8 Test manuel de la Phase 15

6.8.1 Question avec Celery

```
RESPONSE=$(curl -fsS -X POST http://localhost:8000/api/v1/workspaces/
  $WORKSPACE_ID/chat/query \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "question": "How does Celery process documents asynchronously?",
    "retrieval_strategy": "hybrid"
  }')

echo "$RESPONSE" | jq '{answer,citations,evaluation,message_id}'
```

6.8.2 Vérification des chunks utilisés

```
MESSAGE_ID=$(echo "$RESPONSE" | jq -r .message_id)

docker compose exec postgres psql -U devpilot -d devpilot -c \
"select retrieval_strategy, score, rank
from retrieved_chunks
where message_id = '$MESSAGE_ID'
order by rank;"
```

6.8.3 Vérification de l'agent reranker

```
docker compose exec postgres psql -U devpilot -d devpilot -c \
"select agent_type, status, latency_ms
from agent_runs
where message_id = '$MESSAGE_ID'
order by created_at;"
```

Résultat attendu :

```
router
query_rewriter
retrieval
reranker
generator
evaluator
```



```
corrector
```

6.8.4 Vérification de la sortie du reranker

```
docker compose exec postgres psql -U devpilot -d devpilot -c \  
"select agent_type, output  
from agent_runs  
where message_id = '$MESSAGE_ID'  
and lower(agent_type) like '%rerank%'  
order by created_at;"
```

6.9 Validation de la Phase 15

La Phase 15 est validée si :

- le reranker est exécuté ;
- l'agent `reranker` est sauvegardé dans `agent_runs` ;
- les chunks rerankés sont utilisés pour la génération ;
- la réponse finale est correcte ;
- la recherche hybride reste fonctionnelle ;
- les latences sont enregistrées ;
- les features du reranking sont visibles dans la sortie de l'agent.

7 Résultats techniques observés

7.1 État des phases

Phase	État	Preuve de validation
Phase 12	Validée	Évaluations retournées et sauvegardées dans PostgreSQL.
Phase 13	Validée	Exécutions des agents sauvegardées dans <code>agent_runs</code> .
Phase 14	Validée	Recherche hybride fonctionnelle et stratégie sauvegardée.
Phase 15	Validée	Reranker exécuté, scores et features enregistrés.

7.2 Latences observées

Les latences observées montrent que les étapes les plus coûteuses sont :

- la génération avec Ollama ;
- l'évaluation avec Ollama ou fallback ;
- la recherche et le reranking restent rapides.

Exemple :

```
generator average latency: about 54 seconds
evaluator average latency: about 42 seconds
reranker average latency: about 2 ms
retrieval average latency: about 347 ms
```

7.3 Analyse des performances

La lenteur du générateur est principalement due au modèle local utilisé :

```
qwen3:8b
```

Ce modèle donne de meilleurs résultats, mais demande plus de ressources.

Pour le développement quotidien, il est possible d'utiliser :

```
qwen2.5:3b
```

Pour la démonstration finale, `qwen3:8b` reste un bon choix si la machine le supporte.

8 Pourquoi ces technologies sont utilisées

8.1 FastAPI

FastAPI est utilisé pour construire l'API backend. Il offre :

- de très bonnes performances ;
- une documentation Swagger automatique ;
- une excellente intégration avec Python moderne ;
- un support asynchrone.

8.2 PostgreSQL

PostgreSQL est utilisé pour stocker les données relationnelles :

- utilisateurs ;
- workspaces ;
- documents ;
- chunks ;
- conversations ;
- messages ;
- évaluations ;
- exécutions d'agents.

8.3 Qdrant

Qdrant est utilisé comme base vectorielle pour la recherche sémantique.

Il permet :

- le stockage d'embeddings ;
- la recherche de similarité ;
- le filtrage par payload ;
- l'isolation par workspace.

8.4 Ollama

Ollama permet d'utiliser des modèles locaux.

Avantages :

- pas de dépendance obligatoire à OpenAI ;
- réduction des coûts ;
- meilleure confidentialité ;
- contrôle sur les modèles ;
- possibilité de développement local.

8.5 Redis et Celery

Redis et Celery permettent le traitement asynchrone.

Celery exécute les tâches longues, par exemple :

- extraction de texte ;
- chunking ;
- génération d'embeddings ;
- indexation dans Qdrant.

Redis sert de broker de messages entre le backend et les workers.

8.6 Recherche hybride

La recherche hybride est utilisée parce que la recherche sémantique seule n'est pas toujours suffisante.

Elle combine :

- la compréhension du sens ;
- la précision des mots-clés.

8.7 Reranking

Le reranking améliore la qualité du contexte. Il permet d'éviter d'envoyer au modèle des chunks peu utiles, même s'ils ont été récupérés au départ.

9 Commandes de démonstration

9.1 Tester l'évaluation

```
RESPONSE=$(curl -fsS -X POST http://localhost:8000/api/v1/workspaces/  
  $WORKSPACE_ID/chat/query \  
  -H "Authorization: Bearer $TOKEN" \  
  -H "Content-Type: application/json" \  
  -d '{"question":"What technologies does DevPilot AI use?"}')  
  
echo "$RESPONSE" | jq '{answer,citations,evaluation,message_id}'
```

9.2 Tester les agents

```
MESSAGE_ID=$(echo "$RESPONSE" | jq -r .message_id)  
  
docker compose exec postgres psql -U devpilot -d devpilot -c \  
"select agent_type, status, latency_ms  
from agent_runs  
where message_id = '$MESSAGE_ID'  
order by created_at;"
```

9.3 Tester la recherche hybride

```
curl -X POST http://localhost:8000/api/v1/workspaces/$WORKSPACE_ID/retrieval/  
  search \  
  -H "Authorization: Bearer $TOKEN" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "query": "How does DevPilot use Qdrant and Celery?",  
    "top_k": 5,  
    "strategy": "hybrid"  
  }'
```

9.4 Tester le reranking

```
RESPONSE=$(curl -fsS -X POST http://localhost:8000/api/v1/workspaces/  
  $WORKSPACE_ID/chat/query \  
-H "Authorization: Bearer $TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "question": "How does Celery process documents asynchronously?",  
  "retrieval_strategy": "hybrid"  
}')  
  
MESSAGE_ID=$(echo "$RESPONSE" | jq -r .message_id)  
  
docker compose exec postgres psql -U devpilot -d devpilot -c \  
"select agent_type, output  
from agent_runs  
where message_id = '$MESSAGE_ID'  
and lower(agent_type) like '%rerank%'";"
```

10 Difficultés rencontrées et améliorations

10.1 Problème du corrector trop agressif

Durant les tests, le corrector a parfois remplacé une réponse correcte par :

I could not find this information in the uploaded documents.

Cela arrivait lorsque le score de pertinence heuristique était trop faible.

La correction a consisté à rendre le corrector moins agressif :

- ne pas remplacer une réponse si le contexte est bon ;
- ne pas remplacer une réponse si le risque d'hallucination est faible ;
- améliorer la pertinence heuristique pour les questions techniques.

10.2 Fallback heuristique

L'évaluation utilise parfois un fallback heuristique lorsque l'évaluation LLM n'est pas disponible.

Ce fallback est utile, mais il peut être amélioré.

Améliorations possibles :

- meilleure analyse du recouvrement question-réponse-contexte ;
- détection plus intelligente des termes techniques ;
- évaluation séparée des citations ;
- appel à un modèle plus petit dédié à l'évaluation.

10.3 Latence du modèle

L'utilisation de `qwen3:8b` améliore la qualité, mais augmente la latence.

Améliorations possibles :

- utiliser un modèle plus léger en développement ;
- limiter le nombre de tokens générés ;
- réduire la taille du contexte ;
- mettre en cache certaines évaluations ;
- utiliser le streaming pour l'interface utilisateur.

11 Conclusion

Les phases 12 à 15 ont transformé DevPilot AI en une plateforme RAG avancée.

La Phase 12 a ajouté l'évaluation automatique des réponses. La Phase 13 a introduit un workflow agentique traçable. La Phase 14 a amélioré la recherche documentaire grâce à la recherche hybride. La Phase 15 a ajouté un reranking permettant de sélectionner les meilleurs chunks avant la génération.

À ce stade, DevPilot AI ne se limite plus à répondre à des questions. Le système est maintenant capable de :

- récupérer des informations pertinentes ;
- combiner plusieurs stratégies de recherche ;
- réordonner les résultats ;
- générer une réponse sourcée ;
- évaluer la qualité de la réponse ;
- corriger les réponses insuffisantes ;
- tracer toutes les étapes internes.

Cette progression donne au projet une dimension professionnelle, proche d'une véritable plateforme d'IA documentaire sécurisée.

État actuel : Phases 12 à 15 validées.

Prochaine étape : Phase 16 – Mise en place du frontend Next.js.